



CSE 4802

Lab 3

Transformation Tools

Transformation Tools



Def: A transformation tool converts programs between different forms of representations, usually between text and graphics.

For example,

- Transforming code to visual form and vice versa

Because of the impact that visual representations can have on comprehension, use of a transformation tool can help the maintainer view and understand the system in a way that would not be possible with only source code.

Loguru



Loguru is a library which aims to bring enjoyable logging in Python.

Github link: <https://github.com/Delgan/loguru>

Command to install: **pip install loguru**

Features:

- One core object to handle every logging configuration and usage pass through to one of **Logger**'s method
- One function to rule them all
- Easier file logging
- Exceptions catching within threads or main
- Logging with colors
- Fully descriptive exceptions

Loguru



To log a program with loguru, the only object needs to be exported is **logger**

Example:

```
from loguru import logger
```

Most used methods are:

- `logger.trace("A trace message.")`
- `logger.debug("A debug message.")`
- `logger.info("An info message.")`
- `logger.success("A success message.")`
- `logger.warning("A warning message.")`
- `logger.error("An error message.")`
- `logger.critical("A critical message.")`

Loguru



```
2024-02-11 23:21:17.523 | DEBUG | __main__:<module>:8 - A debug message.
2024-02-11 23:21:17.523 | INFO | __main__:<module>:9 - An info message.
2024-02-11 23:21:17.524 | SUCCESS | __main__:<module>:10 - A success message.
2024-02-11 23:21:17.524 | WARNING | __main__:<module>:11 - A warning message.
2024-02-11 23:21:17.524 | ERROR | __main__:<module>:12 - An error message.
2024-02-11 23:21:17.524 | CRITICAL | __main__:<module>:13 - A critical message.
```

Loguru



To add different labels, formatting and level to the logging output to any file, logger provides function called `add()`

Example:

```
logger.add("a.log", format="{time:MMMM D, YYYY > HH:mm:ss!UTC} | {level} | {message}")
```

For serialized logging:

```
logger.add("a.log", format="{time:MMMM D, YYYY > HH:mm:ss!UTC} | {level} | {message}", serialize=True)
```

Loguru always log with all variables changing values, but for sensitive information,

```
logger.add("a.log", format="{time:MMMM D, YYYY > HH:mm:ss!UTC} | {level} | {message}", diagnose=False)
```

Loguru



To add extra information or contextualize the logging information,

```
logger.add(sys.stderr,  
format="{time:MMMM D, YYYY > HH:mm:ss} |  
{level} | {message} | {extra}")  
@logger.contextualize(seller_id="001",  
product_id="123")  
def log():  
    logger.info("A user requested a  
service.")  
log()
```

```
logger = logger.bind(seller_id="001",  
product_id="123")
```

Output:

```
February 11, 2024 > 22:54:06 | INFO | A user requested a service. | {'seller_id': '001', 'product_id':  
'123'}
```

Loguru



To replace the try..catch block, it has @logger.catch() with its own formatting,

Example,

```
@logger.catch(level='CRITICAL', message="Not Good")
```

```
def func1(num1: int, num2: int):
```

```
    return num1/num2
```

```
func1(2,0)
```


PySnooper



PySnooper is a poor man's debugger. It is a Python package that prints the lines of code being executed along with the values of each variable by adding only one decorator.

Github Link: <https://github.com/cool-RR/PySnooper>

Install: `pip install pysnooper`

PySnooper

```
import pysnooper

@pysnooper.snoop()
def number_to_bits(number):
    if number:
        bits = []
        while number:
            number, remainder = divmod(number, 2)
            bits.insert(0, remainder)
        return bits
    else:
        return [0]

number_to_bits(6)
```

PySnooper



```
Source path:... /PySnooper/example.py
Starting var:... number = 6
20:11:56.124936 call          6 def number_to_bits(number):
20:11:56.125577 line          7     if number:
20:11:56.125701 line          8         bits = []
New var:..... bits = []
20:11:56.125787 line          9         while number:
20:11:56.125956 line         10             number, remainder = divmod(number, 2)
Modified var:... number = 3
New var:..... remainder = 0
20:11:56.126042 line         11                 bits.insert(0, remainder)
Modified var:... bits = [0]
20:11:56.126314 line          9         while number:
20:11:56.126494 line         10             number, remainder = divmod(number, 2)
Modified var:... number = 1
Modified var:... remainder = 1
20:11:56.126623 line         11                 bits.insert(0, remainder)
Modified var:... bits = [1, 0]
20:11:56.126870 line          9         while number:
20:11:56.127025 line         10             number, remainder = divmod(number, 2)
Modified var:... number = 0
20:11:56.127161 line         11                 bits.insert(0, remainder)
Modified var:... bits = [1, 1, 0]
20:11:56.127404 line          9         while number:
20:11:56.127621 line         12             return bits
20:11:56.127738 return         12             return bits
Return value:... [1, 1, 0]
Elapsed time: 00:00:00.003039
```

PySnooper



Pysnooper for specific portion

```
def foo():  
    lst = []  
    for i in range(10):  
        lst.append(random.randrange(1, 1000))  
  
    with pysnooper.snoop():  
        lower = min(lst)  
        upper = max(lst)  
        mid = (lower + upper) / 2  
        print(lower, mid, upper)
```

PySnooper



For logging the outputs in a file,

```
@pysnooper.snoop('/my/log/file.log')
```

For without all machine related data,

```
@pysnooper.snoop(normalize=True)
```

For multi-thread informations,

```
@pysnooper.snoop(thread_info=True)
```

Profiling Tools

Code profiling is a method that is used to detect how long each function or line of code takes to run and how often it gets executed.

Most important package for python code profiling is called cProfile:

```
python -m cProfile test.py
```

Results can be seen in terminal like:

```
12 function calls in 0.000 seconds
Ordered by: standard name
ncalls tottime pcall cumtime pcall filename:lineno(function)
  1 0.000 0.000 0.000 0.000 test.py:1(<module>)
  6 0.000 0.000 0.000 0.000 test.py:1(print_method)
  3 0.000 0.000 0.000 0.000 test.py:5(test_print_method)
  1 0.000 0.000 0.000 0.000 {method 'disable' of '_lsprof.Profiler' objects}
  1 0.000 0.000 0.000 0.000 {range}
```

Viztracer



VizTracer is a tool to help you understand Python code by tracing and visualizing its execution. Without making any changes to your source code.

Command to install: `pip install viztracer`

Website link: <https://github.com/gaogaotiantian/viztracer>

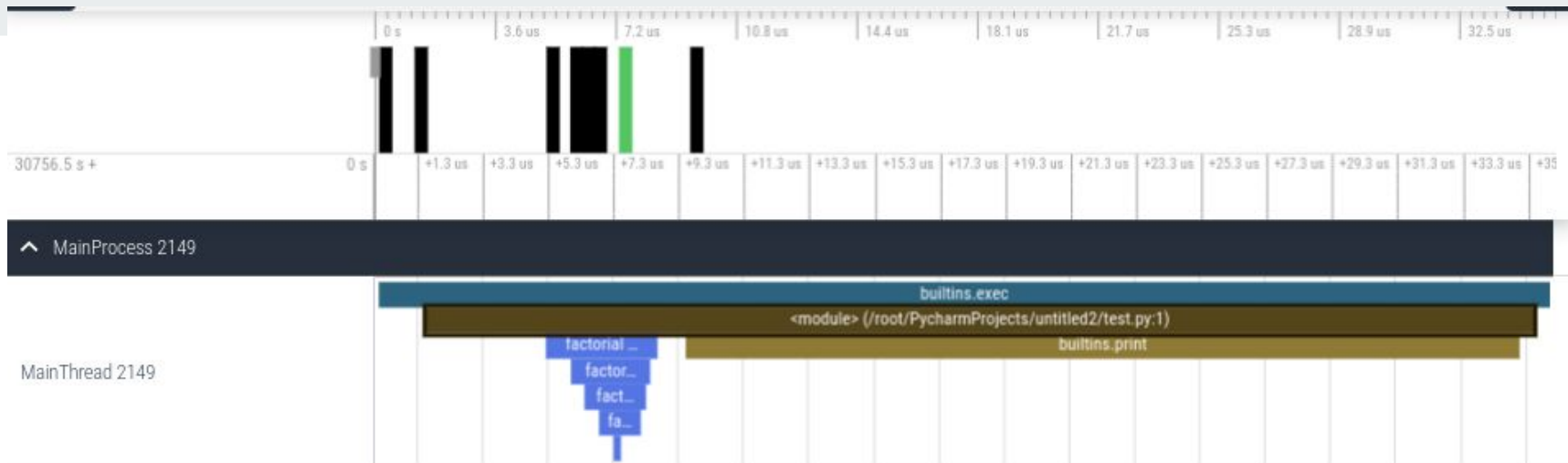
Viztracer



```
def factorial(x):  
    if x == 1:  
        return 1  
    else:  
        return (x * factorial(x-1))  
  
num = 7  
  
result = factorial(num)  
print("The factorial of", num, "is", result)
```

Command in terminal:

```
viztracer -o result.html --open my_script.py
```

Current Selection

Slice Details

Name	<module> (/root/PycharmProjects/untitled2/test.py:1)
Category	FEE
Start time	1us 502ns
Duration	34us 73ns
Slice ID	1
type	thread_slice
track_id	0
depth	1

```

1 def factorial(x):
2     if x == 1:
3         return 1
4     else:
5         return (x * factorial(x-1))
6
7
8 if __name__ == "__main__":
9     num = 5

```

SnakeViz



SnakeViz is a browser based visualization tool. It needs the output in a .profile format.

Install the package:

```
pip install snakeviz --user
```

To Generate the output, two steps to follow:

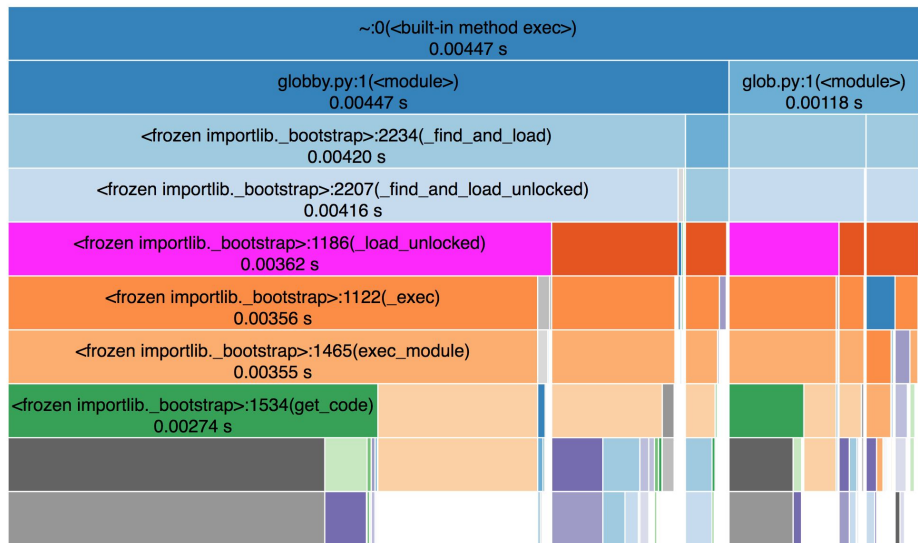
- Generate .profile file:

```
python -m cProfile -o <output_file>.profile <your_file>.py
```
- Visualize profiling information:

```
snakeviz <output_file>.profile
```

SnakeViz

Icicle Chart:



Sunburst Chart:

